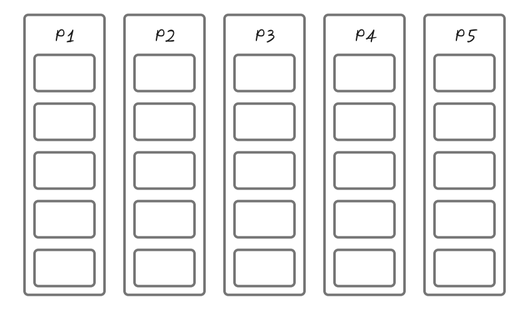
Escribir un programa en el que se manejen un total de n = 5 pilas: P1, P2, P3, P4 y P5. La entrada de datos serán pares de enteros (i,j) tal que 1 ≤ abs(i) ≤ n. De tal forma que el criterio de selección de pila será:
- Si i es positivo, debe insertarse el elemento j en la pila Pi.
- Si i es negativo, debe eliminarse el elemento j de la pila Pi.
- Si i es cero, fin del proceso de entrada.

Los datos de entrada se introducen por teclado. Cuando termina el proceso el programa debe escribir el contenido de la n Pilas en pantalla.

Creamos un arreglo de 5 Pilas

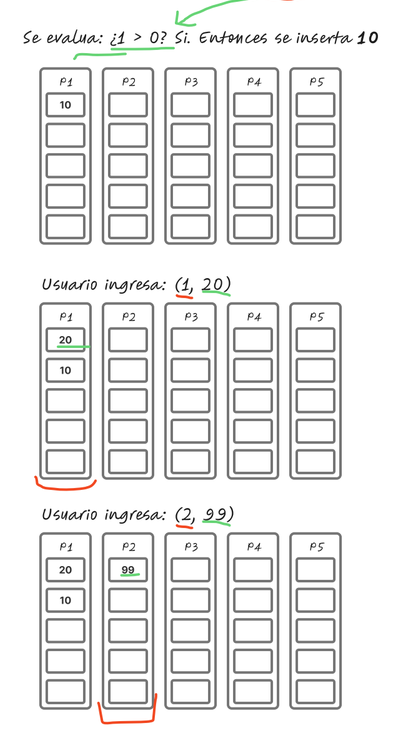
Como los indices en Java van de 0 a 4, Entonce la pila Pi estaria representando el indice abs(i) - 1



Si i > 0 Entonces insertamos el elemento j en la Pi ← Numero de Pila que el usuario ingrese (supongo)

Ejemplo: El usuario ingresa (1, 10)

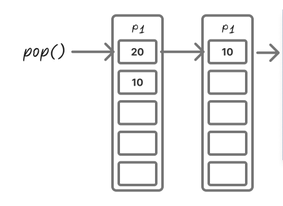
Se evalua: ¿1 > 0? Si. Entonces se inserta **10**



Si i < 0 entonces eliminamos un elemento de la Pi.

Usuario ingresa: (-1, 0)

Se evalua ¿-1 < 0? Si. Entonces eliminar



Es importante verificar previamente que la pila no este vacia para evitar excepciones

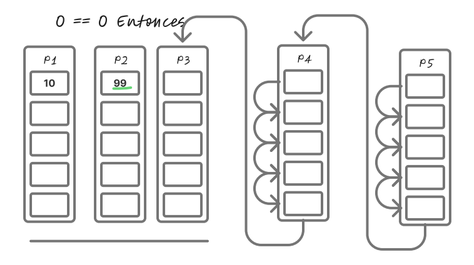
Jaider Rosero

Si i == 0 finalizaria el bucle de lectura

Y por ultimo Se recorreria el arreglo de pilas y se imprime el contenido de las mismas

Ejemplo usuario ingresa (0, 0)

0 == 0 Entonces



Importante

Los arreglos en Java verifican sus tipos de datos en tiempo de ejecución, mientras que los genéricos como en este caso Stack<Integer> utiliza un mecanismo llamado Type Erasure (Borrado de tipos) donde los tipos se eliminan tras compilar.

Al intentar hacer un arreglo de una clase generica como Stack, el compilador lanza una advertencia indicando que no puede verificar totalmente la seguridad de tipos.

Para solucionar esto podemos utilizar la anotacion **@SuppressWarnings("unchecked")**

Con esta anotacion le damos la intruccion directa al compilador de Java que ignore la advertencia (warning) en este caso las advertencias relativas a operaciones no verificadas (unchecked operations)

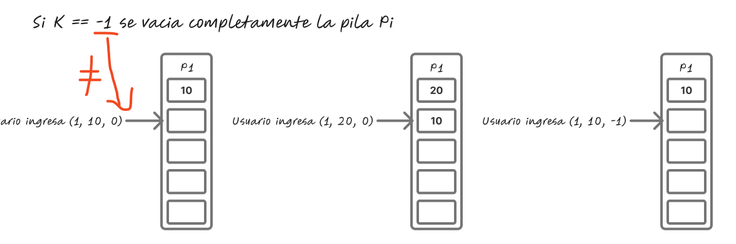
Lo colocamos encima del constructor correspondiente, entonces este le indicaria al compilador Sé que esta línea genera un aviso de tipos, pero el código está controlado y es seguro; no muestres el warning en consola.

Problema PILAS 9.4

~~Modificar el programa del Problema 9.4 para que la entrada sean triplos de números enteros (i,j,k), donde i, j tienen el mismo significado que en 9.4, y k es un número entero que puede tomar los valores −1, 0 con este significado:~~
- ~~-1, hay que borrar todos los elementos de la pila.~~
- ~~0, el proceso es el indicado en el Problema 9.4 con i y j.~~

Sigue la misma logica pero se agrega la evaluacion de la variablre K

Si K == -1 se vacia completamente la pila Pi


≠
Usuario ingresa (1, 10, 0)
P1
10
Usuario ingresa (1, 20, 0)
P1
20
10
Usuario ingresa (1, 10, -1)
P1
10

Problema PILAS 9.5

Se quieren determinar las frases que son palíndromo, para lo cual se ha de seguir la siguiente estrategia: considerar cada línea de una frase; añadir cada carácter de la frase a una pila y, a la vez, a lista enlazada circular por el final; extraer carácter a carácter, simultáneamente de la pila y de la lista circular el primero, su comparación determina si es palíndromo o no. Escribir un programa que lea líneas y determine si son palíndromo.

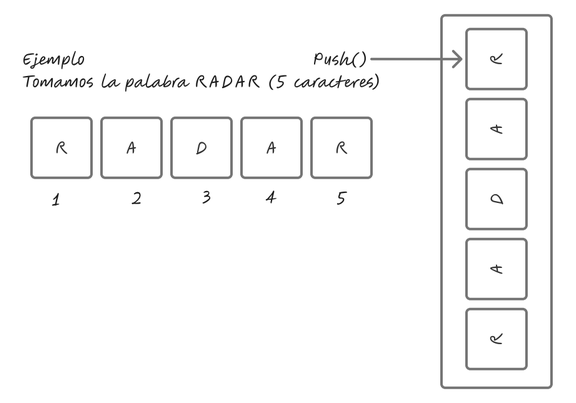
Al insertar los caracteres uno a uno, la pila los almacena de modo que al extraerlos salen en orden inverso

Podemo utilizar una lista enlazada Circular que al insertar cada caracter al final de la lista circular, mantenemos la referencia al primer elemento permitiendo extraer en orden directo los elementos.

Entonces al extraer al mismo tiempo un caracter de la pila y un caracter del inicio de la lista circular hacemos:
- Si la cadena es un polindromo, el primer caracter coincidira con el ultimo, el Segundo con el penultimo, y asi sucesivamente
- Si en algun punto los caracteres extraidos son distintos la frase no es un polindromo

Ejemplo

Tomamos la palabra RADAR (5 caracteres)



Top → R

A

D

A

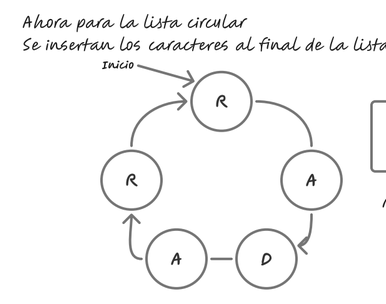
R

Al hacer pop(), los caracteres saldrian en orden inverso:

R→A→D→A→R

Ahora para la lista circular

Se insertan los caracteres al final de la lista circular

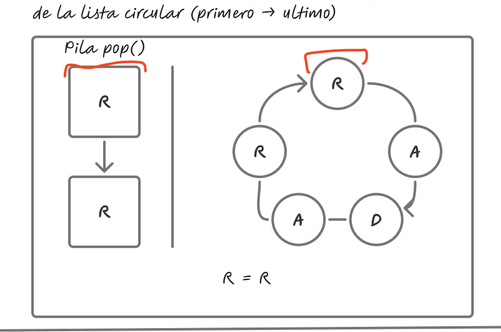


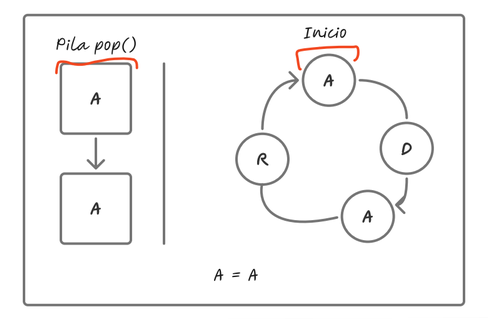
R→A→D→A→R

Mantiene la referencia al primer elemento (inicio)

Comparacion

Se extrae el caracter de la pila (ultimo → primero) y uno de la lista circular (primero → ultimo)



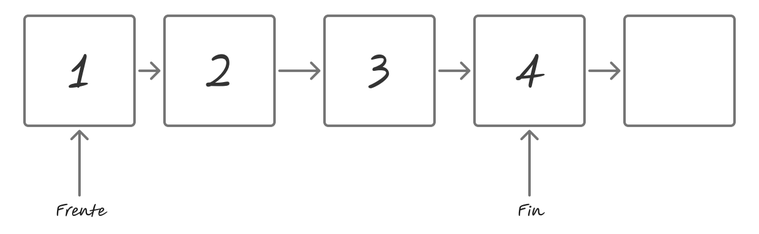


Asi sucesivamente hasta determinar que la palabra RADAR es un polindromo

---

CONCEPTO DE COLA

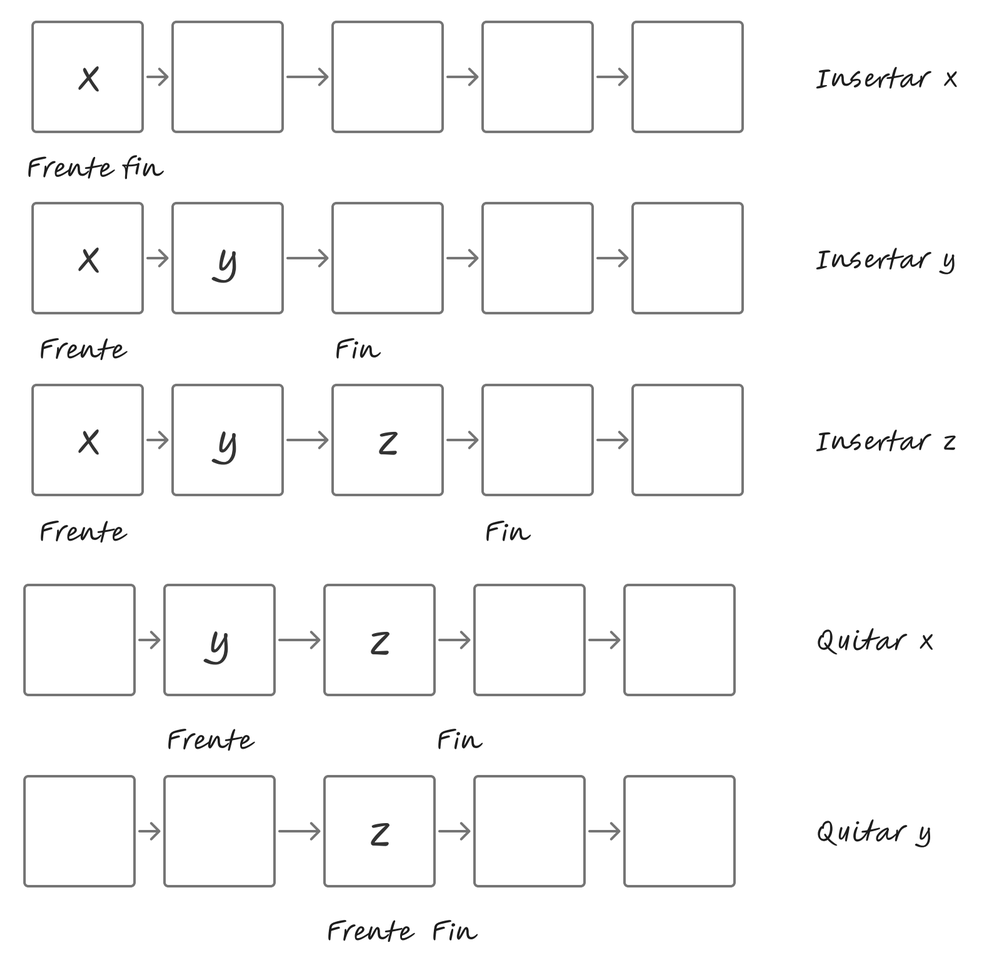
Una cola es una estructura de datos que almacena elementos en una lista y permite acceder a los datos por uno de los dos extremos de la lista . Un elemento se inserta en la cola (parte final) de la lista y se suprime o elimina por el frente (parte inicial, frente) de la lista. Las aplicaciones utilizan una cola para almacenar elementos en su orden de aparición o concurrencia.



Los elementos se eliminan (se quitan) de la cola en el mismo orden en que se almacenan y, por consiguiente, una cola es una estructura de tipo FIFO (first-in, firs-out, primero en entrar-primero en salir o bien primero en llegar-primero en ser servido). El servicio de atención a clientes en un almacén es un ejemplo típico de cola. La acción de gestión de memoria intermedia (buffering) de trabajos o tareas de impresora en un distribuidor de impresoras (spooler) es otro ejemplo típico de cola . Dado que la impresión es una tarea (un trabajo) que requiere más tiempo que el proceso de la transmisión real de los datos desde la computadora a la impresora, se organiza una cola de trabajos de modo que los trabajos se imprimen en el mismo orden en el que se recibieron por la impresora. Este sistema tiene el gran inconveniente de que si su trabajo personal consta de una única página para imprimir y delante de su petición de impresión existe otra petición para imprimir un informe de 300 páginas, deberá esperar a la impresión de esas 300 páginas antes de que se imprima su página.

En una cola, al igual que en una pila, los datos se almacenan de un modo lineal y el acceso a los datos sólo está permitido en los extremos de la cola.

La forma que los lenguajes tienen para representar el TAD Cola depende de donde se almacenen los elementos: en un array, en una estructura dinámica como puede ser un Vector (contenedor de Java) o en una lista enlazada. La utilización de arrays tiene el problema de que la cola no puede crecer indefinidamente, está limitada por el tamaño del array; como contrapartida, el acceso a los extremos es muy eficiente. Utilizar una lista enlazada permite que el número de nodos se ajuste al de elementos de la cola; por el contrario, cada nodo necesita memoria extra para el enlace, y también hay que tener en cuenta el límite de memoria de la pila del computador.

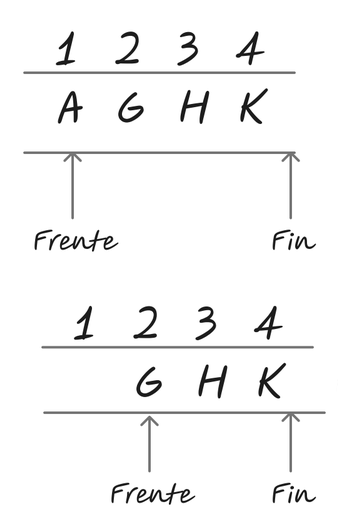


COLAS IMPLEMENTADAS CON ARRAYS

Al igual que las pilas, las colas se implementan utilizando una estructura estática (arrays) o una estructura dinámica (listas enlazadas, Vector...). En esta sección se considera la implementación con arrays. La declaración de una Cola ha de contener un array para almacenar los elementos de la cola y dos marcadores o apuntadores para mantener las posiciones frente y fin de la cola, es decir, un marcador apuntando a la posición de la cabeza de la cola y el otro al primer espacio vacío que sigue al final de la cola. Cuando un elemento se añade a la cola, se verifica si el marcador fin apunta a una posición válida, y entonces se añade el elemento a la cola y se incrementa el marcador fin en 1. Cuando un elemento se elimina de la cola, se hace una prueba para ver si la cola está vacía y, si no es así, se recupera el elemento de la posición apuntada por el marcador de cabeza, y éste se incrementa en 1.

La operación de poner un elemento en la cola comienza en la posición fin 0, y cada vez que se añade un nuevo elemento se incrementa fin en 1. La extracción de un elemento se hace por el extremo contrario, frente, y cada vez que se extrae un elemento avanza frente una posición. Se puede observar cómo avanza el puntero frente al extraer un elemento.

El avance lineal de frente y fin tiene un grave problema, deja huecos por la izquierda del array . Llega a ocurrir que fin alcanza el índice mas alto del array, sin que puedan añadirse nuevos elementos, y, sin embargo, hay posiciones libres a la izquierda de frente.



Posicion frente y fin despues de extraer

Una alternativa consiste en mantener fijo el frente de la cola al comienzo del array y mover todos los elementos de la cola una posición cada vez que se retira un elemento. Estos problemas quedan resueltos considerando el array como circular.

DECLARACION DE LA CLASE COLA

Una cola debe tratar elementos de diferentes tipos de datos: enteros, cadenas, objetos... Por esta circunstancia, los elementos han de ser de un tipo genérico. En Java todavía no está definido el tipo genérico; sin embargo, declarando el tipo del elemento Object y utilizando objetos y referencias se puede conseguir el mayor grado de genericidad.

La clase ColaLineal contiene un array (listaCola) cuyo máximo tamaño se determina por la constante MAXTAMQ. El tipo de los elementos queda sin especificar (TipoDeDato), para que pueda sustituirse por un tipo simple, o bien por Object. Los atributos frente y fin son los punteros de cabecera y cola (fin), respectivamente. El constructor de la clase inicializa una cola vacía y define el array: new TipoDeDato [MAXTAMQ].

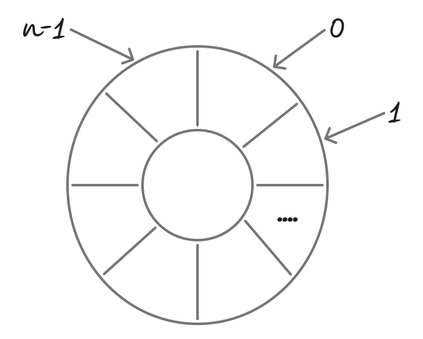
La realización de una cola con un array lineal es notablemente ineficiente, se puede alcanzar la condición de cola llena existiendo elementos del array sin ocupar. Se aconseja no utilizar esta implementación.

Jaider Rosero

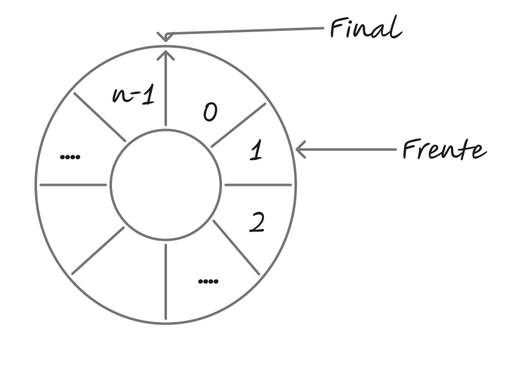
COLA CON UN ARRAY CIRCULAR

La alternativa, sugerida en la operación de quitar un elemento, de desplazar los restantes elementos del array de modo que la cabeza de la cola vuelva al principio del array, es costosa en términos de tiempo de computadora, especialmente si los datos almacenados en el array son estructuras de datos grandes.

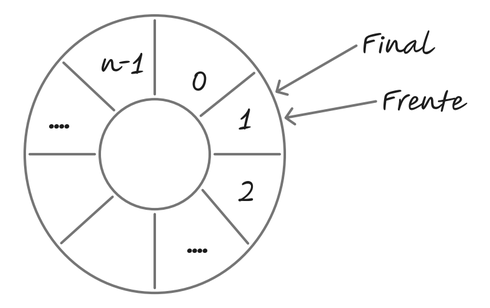
La forma más eficiente de almacenar una cola en un array es modelarlo de tal forma que se una el extremo final con el extremo cabeza. Tal array se denomina array circular y permite que la totalidad de sus posiciones se utilicen para almacenar elementos de la cola sin necesidad de desplazar elementos.



El array se almacena de modo natural en la memoria como un bloque lineal de n elementos. Se necesitan dos marcadores (apuntadores) frente y fin para indicar, respectivamente, la posición del elemento cabeza y del último elemento puesto en la cola.



El frente siempre contiene la posición del primer elemento de la cola y avanza en el sentido de las agujas del reloj; fin contiene la posición donde se puso el último elemento y también avanza en el sentido del reloj (circularmente a la derecha). La implementación del movimiento circular se realiza según la teoría de los restos, de tal forma que se generen índices de 0 a MAXTAMQ-1:

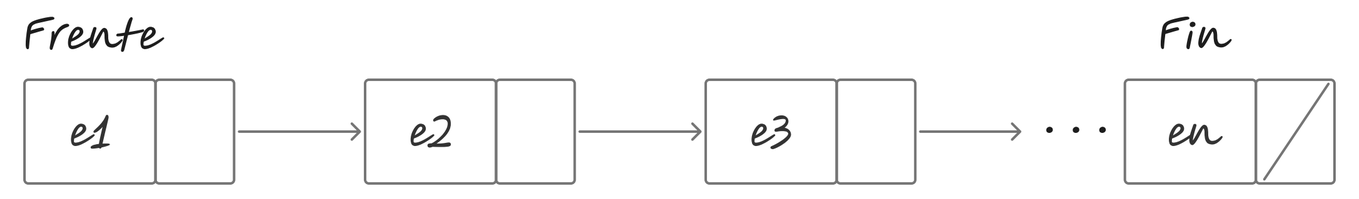


Los algoritmos que formalizan la gestión de colas en un array circular han de incluir las operaciones básicas del TAD Cola, en concreto, las siguientes tareas básicas:
- Creación de una cola vacía, de tal forma que fin apunte a una posición inmediatamente anterior a frente:
  - frente = 0; fin = MAXTAMQ-1
- Comprobar si una cola esta vacia
  - frente == siguiente(fin)
- Comprobar si una cola está llena. Para diferenciar la condición de cola llena de cola vacía se sacrifica una posición del array, de tal forma que la capacidad de la cola va a ser MAXTAMQ-1. La condición de cola llena es:
  - frente == siguiente(siguiente(fin))
- Poner un elemento a la cola: si la cola no está llena, avanzar fin a la siguiente posición, fin = (fin + 1) % MAXTAMQ, y asignar el elemento.
- Retirar un elemento de la cola: si la cola no está vacía, quitarlo de la posición frente y avanzar frente a la siguiente posición:(frente + 1) % MAXTAMQ.
- Obtener el elemento primero de la cola, si la cola no está vacía, sin suprimirlo de la cola.

COLA CON UNA LISTA ENLAZADA

La implementación del TAD Cola con un array necesita reservar memoria para el máximo de elementos previstos. En muchas ocasiones, esto da lugar a que se desaproveche memoria, pero también puede ocurrir los contrario, que se llene la cola y no se pueda seguir con la ejecución del programa a no ser que se amplíe la capacidad del array. Un diseño alternativo consiste en utilizar una lista enlazada que, en todo momento, tiene el mismo número de nodos que elementos la cola.

La implementación del TAD Cola con una lista enlazada permite ajustarse exactamente al número de elementos de la cola. Utiliza dos apuntadores (referencias) para acceder a la lista, frente y fin, que son los extremos por donde salen y por donde se ponen, respectivamente, los elementos de la cola.



La referencia frente apunta al primer elemento de la lista y, por tanto, de la cola (el primero en ser retirado). La referencia fin apunta al último elemento de la lista y también de la cola. La lista enlazada crece y decrece según las necesidades, según se incorporen o se retiren elementos, entonces, en esta implementación no se considera el control de Cola llena.

PROBLEMAS COLAS JAVA
PROBLEMA 10.3

Un pequeño supermercado dispone en la salida de tres cajas de pago. En el local hay 25 carritos de compra. Escribir un programa que simule el funcionamiento, siguiendo las siguientes reglas:
- Si cuando llega un cliente no hay ningún carrito disponible, espera a que lo haya.

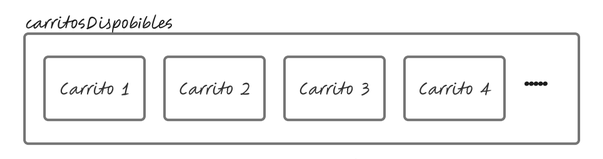
- Ningún cliente se impacienta y abandona el supermercado sin pasar por alguna de las colas de las cajas.

- Cuando un cliente finaliza su compra, se coloca en la cola de la caja que hay menos gente, y no se cambia de cola.

- En el momento en que un cliente paga en la caja, su carrito de la compra queda disponible.

Representar la lista de carritos de la compra y las cajas de salida mediante colas.

Podemos manejar la siguiente estructura



25 carritos

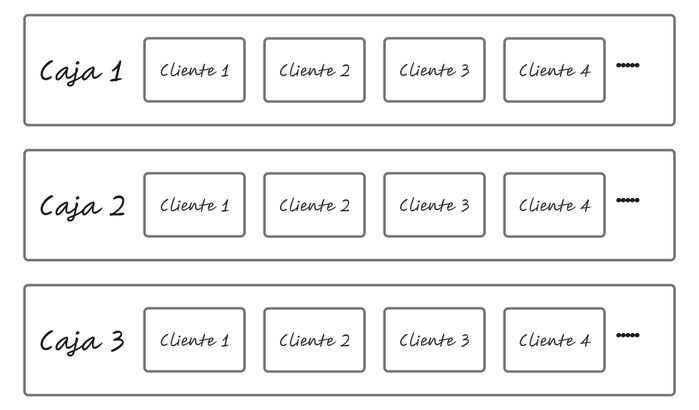
Guardamos los carritos libres en la entrada

colaEsperaCarritos
Cliente 1
Cliente 2
Cliente 3
Cliente 4
....

Clientes en espera

Almacenamos a las personas que llegan cuando los 25 carritos ya esten ocupados.

Creamos un arreglo de 3 colas de cliente

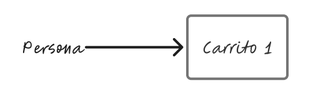


De esta manera representamos las filas físicas de personas esperando por pagar en cada caja.

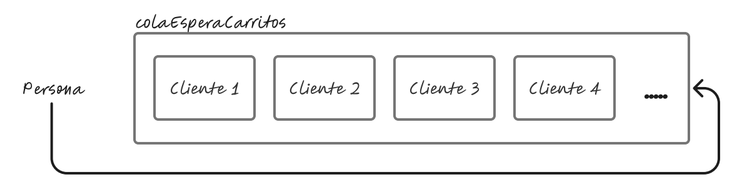
Entonces Entra un nuevo cliente

1. Entra al supermercado y solicita un carro
2. ¿Hay carritos disponibles?

Si. Se extrae un carrito poll(), se le asigna al cliente y este pasa a hacer sus compras



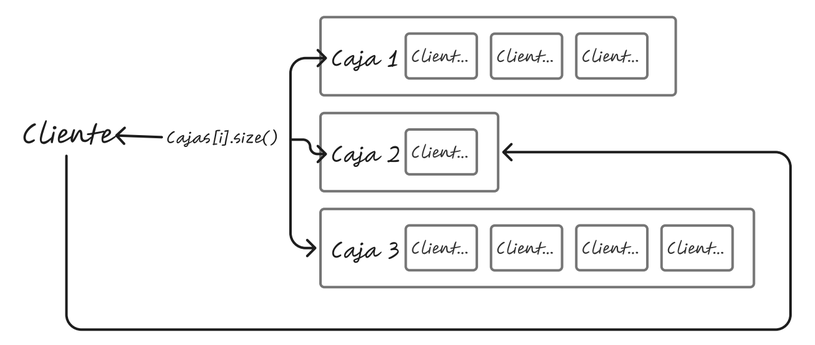
No. El cliente se agrega al final de la cola de espera y espera su turno.



Resultado esperado: el cliente tiene su carrito o queda en espera

Supongamos que ya termino de comprar y ahora quiere elegir la caja

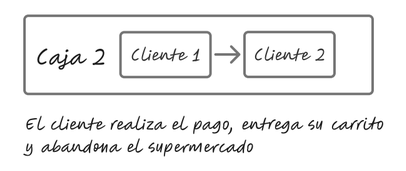
Entonces ahora por logica el cliente deberia hacer cola para terminar su compras pero este debe ver la cola que este mas vacia, es decir la cola donde se encuentren menos personas haciendo fila.
- Comparamos la cantidad de personas en cada cola
- Se une a la caja que tenga menos gente en ese momento
- Permanece en la fila hasta que le toque pagar (por la regla de no cambiarse de caja)



El cliente siempre se une a la caja con menos gente en ese momento

Ahora el cliente esta haciendo la fila en la caja

La caja atiende al primer cliente de su fila

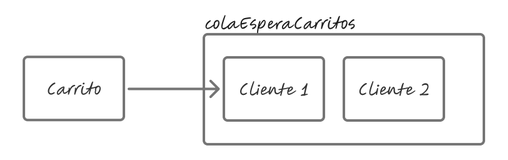


El cliente realiza el pago, entrega su carrito y abandona el supermercado

¿Que ocurre con el carro liberado?

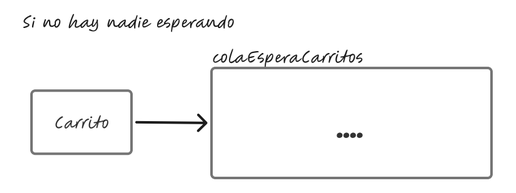
Si hay alguien esperando en ColaEsperaCarritos:

El carrito se entrega al primer cliente de la fila de espera

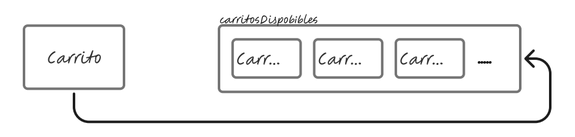


Entonces ahora ese cliente puede ingresar a comprar

Si no hay nadie esperando



El carrito regresa a la cola de carritosDisponibles en la entrada



Entonces tenemos:

1. Cliente entra y pide carrito (cola o carrito disponible)
2. Termina compra y elige caja (menor fila)
3. Paga en caja y entrega carrito (se recicla)

Manteniendo el sistema funcionando de manera ordenada, justa y eficiente

---

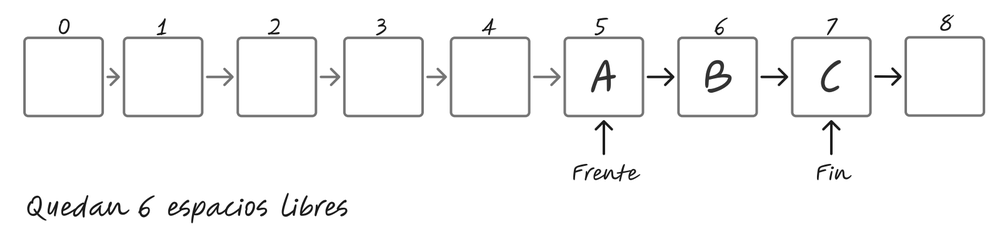
PROBLEMA 10.5

Considere una bicola de caracteres, representada en un array circular. El array consta de 9 posiciones. Los extremos actuales y los elementos de la bicola son:

Frente = 5 fin = 7 Bicola: A,B,C

Escribir los extremos y los elementos de la bicola según se realizan estas operaciones:
- Añadir los elementos F y K por el final de la bicola.
- Añadir los elementos R, W y V por el frente de la bicola.
- Añadir el elemento M por el final de la bicola
- Eliminar dos caracteres por el frente
- Añadir los elementos K y L por el final de la bicola.
- Añadir el elemento S por el frente de la bicola.

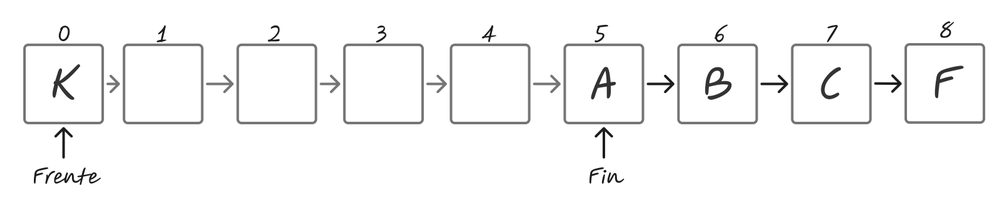
El arreglo tiene 3 elementos. El frente esta en 5 y el fin en 7



Quedan 6 espacios libres

Añadimos 'F' y 'k' por el final para avanzar por la derecha

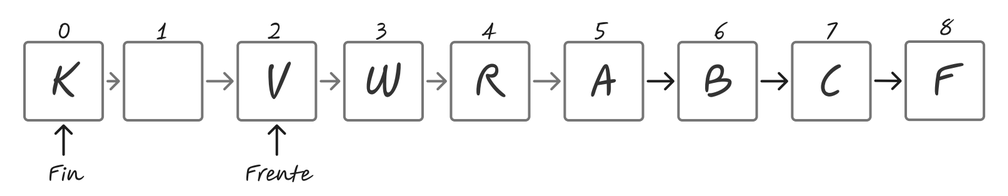
El fin avanza. Para 'F', pasa al 8. Para 'K', como se acaba el arreglo por la derecha, da la vuelta y aparece en el indice 0



Como el fin cruzo el limite derecho y reaparece al inicio

Añadimos 'R', 'W' y 'k' por el frente entoces avanza por izquierda

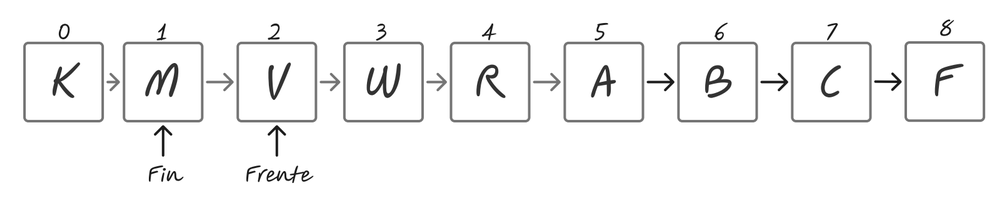
Entonces se mueve del 5 hacia la izquierda pasa al 4 'R', luego al 3 'W' y finalmente al 2 'V'



ahora los datos van desde el indice 2 hasta el 8, continuan en el 0

Añadimos 'M' por el final

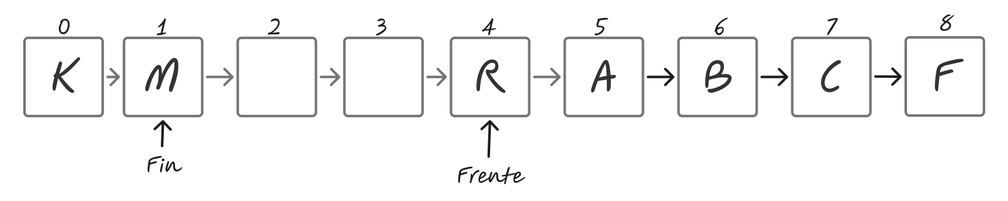
El fin avanza un espacio hacia la derecha (del 0 al 1)



Ahora el arreglo estaria lleno quendando el Fin un paso detras del frente

Ahora eliminar 2 elementos por el Frente

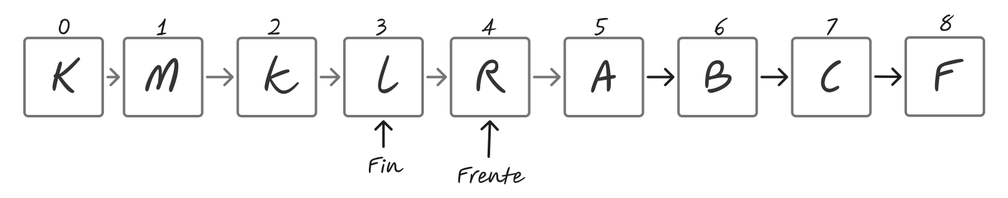
Salen los elementos de la cabeza ('V' y 'W'). Al eliminar por el Frente, este puntero avanza hacia la derecha para apuntar a la nueva cabeza, pasando del 2 al 4.



Se han liberado los espacios 2 y 3. La nueva cabeza es 'R'

Ahora añadir 'K', 'L' por el Final

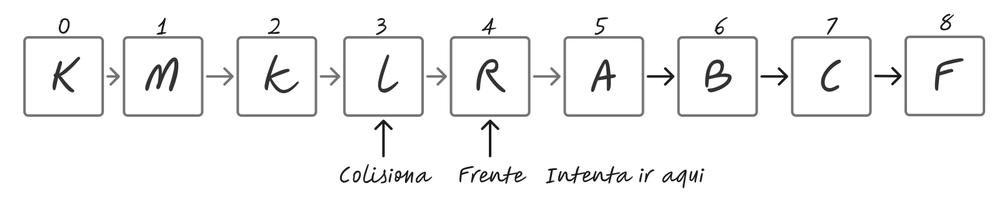
Los nuevos elementos entran por el Fin ocupando el espacio que se acaba de liberar. El Fin avanza hacia la derecha (del 1 al 3).



Arreglo lleno nuevamente Las 9 casillas están ocupadas

Ahora intentamos añadir 'S' por el Frente

Quieres insertar un elemento retrocediendo el Frente un espacio hacia la izquierda (pasando del 4 al 3).



Al intentar mover el frente al índice 3, el sistema detecta que esa casilla ya está siendo ocupada por el último elemento que entró 'L', marcado por el final.

Como la cantidad total de elementos es igual a la capacidad (9 de 9), se rechaza la operación. (Desbordamiento (Overflow))